

How the Git provenance anchor works

A system briefing for the classification review

Isaac Epaphras Nana Sam

6 October 2026

This note describes the system you classified against the Ladisa taxonomy. It is drawn from Chapters 1 and 3 of the thesis. It describes what the system records and what a later check compares. It contains no classification result and no coverage figure. Your original workbook stays as you returned it.

1 The problem

Modern software is assembled from many upstream projects, and an attacker who compromises one widely used project can reach everyone who installs the affected release. Two incidents shape this design.

In SolarWinds (2020), the source repository showed no trace of the attack. The backdoor entered during compilation inside a compromised build environment, so the signed binary differed from what the source would produce.

In XZ Utils (2024), an attacker spent about two years earning commit access. Parts of the backdoor sat in committed test files, but the activation code was present only in the release tarball. The tarball therefore differed from the tagged repository.

A third weakness is the hosting platform. On GitHub or GitLab, a user with enough permission can delete and recreate a tag or rewrite branch history. A verifier who relies only on the hosted repository has to trust the platform to show the same tag later.

The system answers one question: does the state someone receives later still match the state that was recorded when the release was tagged? It is a detective control. It produces a signal when a later check disagrees with the record. It does not block an attack, and it does not judge whether code is benign.

2 What is recorded

When a release tag is pushed, the system records the Git tree hash of the commit that tag points to. Git already computes this value (`git rev-parse <tag>^{tree}`). The tree hash depends on every tracked file, its contents and mode, every subdirectory, and every submodule identifier. If any tracked file changes, the tree hash changes.

The tree hash does not cover tag metadata, commit history, ignored files, files stored outside Git through Git LFS, or build outputs.

The same transaction carries the SHA-256 hash of a CycloneDX software bill of materials (SBOM), generated by Syft from the same tagged tree. Both hashes go to a small smart contract deployed on three Ethereum Layer-2 networks: Arbitrum One, OP Mainnet and zkSync Era. The submitted value has a fixed size, so the on-chain cost does not grow with the repository.

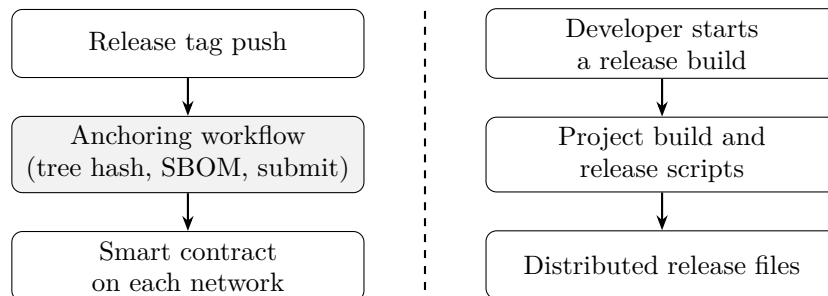
The contract does three things.

- It accepts a submission only from an account on that repository's allowlist. Records are keyed by a project identifier and a tag or reference name, not by a package name.

- It stores the latest record for each reference. A new submission for the same reference replaces what the contract returns and increments a revision number.
- It emits an event for every submission. Events cannot be edited or removed, so earlier submissions stay visible in the chain’s history after a later one replaces the stored value.

3 How anchoring runs

Anchoring runs in its own small CI workflow (for example `.github/workflows/provenance-anchor.yml`). Only a release-tag push starts it. It does five things: check out the tagged state, compute the tree hash, generate the SBOM, submit both hashes, and read the stored record back to confirm it.



The anchoring workflow never calls, imports or depends on the project’s own build or release scripts. Someone who tampers with those scripts cannot change what the anchor computes, because the anchor is computed from Git directly. On GitHub, the workflow file is protected: changes need a pull request and approval from a code owner, with administrator bypass disabled.

Each repository has its own signing key, held in the CI platform’s encrypted secret store, where project build scripts cannot read it. The contract checks only that the transaction carries a valid signature from an allowlisted account. A maintainer can remove a stolen key from the allowlist once the theft is noticed.

This separation has limits. It covers a compromise confined to the project’s build logic. It does not cover a compromise of the CI hosting platform, of the anchoring workflow itself, or of the signing key. If any of those signs a false statement, the contract accepts it.

- *Before any honest anchor exists*, the false statement becomes the record. A later check compares against the false record. The event still shows which account submitted it and when.
- *After an honest anchor exists*, a false submission replaces the value the contract returns. The honest submission remains in the event history.

4 How verification works

A command-line verifier reads the stored record from every configured network and fails if the networks disagree or if any network has no record. It can check three kinds of object.

A Git reference. The verifier resolves the tag again, recomputes the tree hash, and compares it with the record. If the tag has been moved to another commit, or history has been rewritten so that the tagged tree differs, the hashes do not match.

A source release archive. The verifier rebuilds a Git-style tree hash from the archive’s files and compares it with the record. Release archives often contain legitimate generated files that are not in Git, such as a `configure` script. A repository-tracked manifest lists those paths. If the first

comparison fails, the verifier removes the declared paths and compares again. A match after that is a pass with declared extras.

- A changed tracked file, or an extra file at an undeclared path, gives a mismatch.
- The manifest authenticates which paths may be extra, not what they contain. Different content placed at a declared path is not detected.
- A mismatch is reported for the archive as a whole. The verifier does not name the file responsible.

An SBOM file. The verifier hashes a supplied SBOM and compares it with the SBOM hash in the record.

The verifier does not compare compiled binaries with source. A binary built from honest source but altered during compilation gives no mismatch, because nothing ties the binary to the tree. Closing that gap would need reproducible builds or build attestation, which this system does not provide.

5 Three anchoring policies

The system can run in three ways, and they differ in what can be noticed later.

Policy	What it adds	What it does not do
Tag-only anchoring	One anchor when each release tag is pushed.	A moved tag is noticed only if someone happens to compare it with the record.
Tag anchoring with branch snapshots	Periodic anchors of the production branch between releases.	A snapshot records state; it does not compare anything. A bad commit is captured only if it is present when a snapshot runs. Each snapshot costs a transaction.
Tag anchoring with scheduled re-verification	A scheduled job re-resolves existing tags and compares them with their records.	It submits nothing, so it costs no transaction fee. It notices a moved tag within the check interval, not instantly.

6 What lies outside the system

- **Code that was bad before the tag was anchored.** Malicious code merged through review, committed by a compromised maintainer, or hidden in a tracked generated or minified file becomes part of the tree. The anchor records it faithfully, and later checks agree with it.
- **Compiled binaries.** They are not compared with source.
- **Content at declared generated-extra paths,** and files held in Git LFS or other external storage.
- **Compromise of the anchoring workflow, the CI hosting platform or the signing key,** as described in Section 3.
- **Choosing a different package.** Verification is bound to a project identifier. If a consumer installs a look-alike or substituted package from another project, that package has no record under the genuine project, so the genuine record is never contradicted.
- **Forcing delivery.** The public record can show that a newer release was anchored, but the system cannot make a distributor or mirror deliver it.